Literature Survey and Requirement Analysis: A Smart Machine Learning Approach for Phishing Website Detection

Jeevitha L Shetty
SRN: R23EQ045

***Abstract*—This assignment presents a literature survey and requirement analysis for a smart machine-learning approach to phishing website detection. It follows the nine studies summarized in the supplied literature review, covering traditional machine learning, NLP-enhanced and domain-aware features, deep learning, hybrid ensembles, and explainable AI. The studies show that phishing detection has progressed from blacklists and fixed rules to learned URL representations and combined models. They also reveal recurring concerns: performance on unseen websites, false positives and false negatives, limited feature diversity, real-time response, explainability, adaptation to changing attacks, and dataset quality. Based on these findings, the document compares existing approaches, identifies research gaps, and derives functional and non-functional requirements for the proposed module. Reported results are retained as stated in the supplied review; where a metric or dataset detail was not reported, it is not inferred.**

***Index Terms*—phishing detection, machine learning, URL classification, literature survey, requirement analysis, explainable AI.**

I. Introduction

Phishing is a social-engineering and web-security threat in which attackers imitate trusted websites or organizations to obtain credentials, financial information, and other sensitive data. A phishing detection system attempts to identify suspicious websites before users disclose information. The supplied literature review describes a progression from blacklist-based and rule-based detection to traditional machine learning, NLP-enhanced and domain-aware approaches, deep-learning models, hybrid ensembles, and explainable AI.

The purpose of this assignment is to examine existing academic approaches, identify limitations that motivate further work, and derive the functional and non-functional requirements of a smart machine-learning phishing detection module. The review covers URL-only, domain-aware, and webpage/HTML-based signals; these are distinct approaches with different data and implementation requirements.

A. Problem Statement

Blacklist-based systems depend on prior knowledge of malicious URLs and may not recognize newly created sites. Rule-based systems can be understandable but may be evaded by changing URL or page patterns and can produce false positives. Machine-learning systems can learn patterns from labelled examples, but their measured performance depends on dataset quality, selected features, train/test separation, and evaluation metrics. The module therefore needs to evaluate more than a single accuracy figure and should consider generalization, false alarms, missed phishing cases, response time, and explainability.

B. Aim and Objectives

- **Aim:** To specify and evaluate a smart machine-learning module for phishing website detection using evidence from the reviewed literature.
- **Objective 1:** Survey and compare existing blacklist, heuristic, traditional ML, NLP/domain-aware, deep-learning, and hybrid detection approaches.
- **Objective 2:** Identify research gaps relating to generalization, feature diversity, false positives and false negatives, real-time detection, explainability, adaptation, and dataset quality.
- **Objective 3:** Derive functional and non-functional requirements and define an evaluation plan for the proposed module.

II. Survey Scope and Reviewed Studies

This document follows the supplied review, "A Smart Machine Learning Approach for Phishing Website Detection: A Source-Based Review of Nine Studies (2020–2026)." It uses the authors, years, methods, datasets, and reported results as summarized in that document. It does not claim to be a new systematic search of every paper in the field. The supplied review notes that results from different datasets and protocols are not directly comparable, and that NR means "Not Reported."

A. Summary of the Nine Reviewed Studies

Table I summarizes the method, dataset, reported result, and limitation of each reviewed study.

TABLE I

SUMMARY OF THE NINE REVIEWED STUDIES

Study	Method	Dataset	Reported result	Limitation / note
Lakshmanarao, Rao and Bala Krishna (2021)	Traditional ML fusion; ANOVA and mutual-information feature selection; Logistic Regression, Decision Tree, Random Forest, AdaBoost and Gradient Boosting.	UCI phishing dataset: 11,055 instances; 7,738 training and 3,317 testing.	97% accuracy for fusion classifier; other metrics NR.	Only one dataset was used; broader validation was identified as necessary.

Study	Method	Dataset	Reported result	Limitation / note
Gujar (2024)	NLP plus domain features from Python-Whois and DNSPython; AutoML and multiple candidate models.	Dataset name and size NR.	Accuracy 95.8%, precision 94.5%, recall 96.3%, F1 95.4%; AUC NR.	Domain-aware signals add evidence, but dataset details were not reported in the reviewed summary.
Patel, Raj and Sapra (2025)	Decision Tree, SVM, Random Forest, XGBoost, Logistic Regression, MLP and ANN using URL structure, length and suspicious keywords.	5,000 URL samples.	XGBoost accuracy 0.863; DT/RF/XGBoost precision, recall and F1; SVM accuracy 0.797 and Logistic Regression 0.796.	The review identifies tuning, feature engineering, ensembling, scalability and over/underfitting as ongoing issues.
Yaswanth et al. (2026)	Hard-voting ensemble of Logistic Regression, SVM, Random Forest, XGBoost and LSTM; SHAP/LIME, risk scoring and adaptive retraining.	50,000 URLs, balanced 25,000 phishing / 25,000 legitimate; Kaggle, PhishTank and Tranco; 70/15/15 split.	Ensemble accuracy 97.5%, precision 97.3%, recall 97.0%, F1 97.1%.	More operationally complete, but reported figures still need to be interpreted in the context of its particular data and split.
Renjith P N et al. (2025)	TF-IDF character/token representation with a two-layer BiLSTM.	651,191 Kaggle URLs across four classes.	Overall accuracy 90%; phishing-class F1 0.71.	Overall accuracy can conceal weaker performance on the target phishing class.
Sreenivasan, Ramachandran, Rout and Pal (2024)	Tokenization, stemming, normalization and CountVectorizer with Logistic Regression, Linear Regression and Multinomial Naive Bayes.	More than 100,000 labelled URLs.	Optimized Logistic Regression accuracy 98.0369%; precision/recall/F1/AUC NR in the supplied review.	Feature selection, large-scale processing and adaptation to evolving tactics remain challenges.
Parameshwar Patil et al. (2020)	Random Forest, SVM and neural network with backpropagation using URL length, subdomain count and related features.	UCI-related dataset; exact size NR.	Random Forest 97.369%, SVM 97.451%, neural network 97.259% accuracy.	Wrapper-based feature selection adds computational overhead; blacklists struggle with new sites.
Krishnaveni and Balamurugan (2024)	SVC, Random Forest, Decision Tree and XGBoost with feature ranking and dimensionality reduction.	Kaggle phishing URL dataset; exact size NR.	Similarity measure reportedly improved accuracy by 21.8%; final precision/recall/F1 NR.	The reviewed summary does not provide final detailed metrics or exact dataset size.
Kamble and Mishra (2024)	Hybrid deep learning with NLP features and character embeddings; RNN, LSTM, BiLSTM and CNN variants.	Ebbu2017: 36,400 legitimate and 37,175 phishing URLs, plus 26,000 supplementary URLs.	Accuracy 95.79% on first dataset and 97.81% on second; BiLSTM outperformed standard LSTM.	Other detailed metrics were not reported in the supplied review.

III. Thematic Literature Review

A. Blacklist-Based Detection

Blacklist-based detection checks whether a URL or domain is already known to be malicious. As described in the supplied review, the method is simple and potentially fast, but depends on prior discovery and list freshness. A newly created or previously unreported phishing URL may be absent from the list.

B. Rule- and Heuristic-Based Detection

Rule-based systems apply predefined patterns to URLs or pages. Their strengths are simplicity and interpretability. However, attackers may change URL or page structures to evade fixed rules, and the supplied review notes false positives as a limitation.

C. Traditional Machine Learning on URL Features

Traditional classifiers learn from labelled examples using URL characteristics such as length, dots, hyphens, digits, suspicious tokens, subdomain count and IP-address presence. The reviewed studies use Logistic Regression, Decision Trees, Random Forest, SVM, AdaBoost, Gradient Boosting, XGBoost and Naive Bayes. Feature selection and fusion are used to improve performance. Their reported scores vary across datasets, and a single high accuracy does not establish generalization.

D. NLP-Enhanced and Domain-Aware Detection

NLP methods represent URL tokens and text using approaches such as TF-IDF or CountVectorizer. Domain-aware methods add registration and DNS information, for example domain age or nameserver details. Such signals may improve evidence beyond raw URL text, but they require additional data collection and may introduce latency or missing values.

E. Deep Learning

CNN, RNN, LSTM and BiLSTM models learn character or token patterns from URL representations. Hybrid approaches combine NLP features with character embeddings. The supplied review highlights an important caution: Renjith P N et al. (2025) reported 90% overall accuracy but a phishing-class F1 of 0.71. Class-specific precision, recall and F1 should therefore be examined, rather than relying on overall accuracy alone.

F. Hybrid Ensembles and Explainable AI

Ensembles combine predictions from multiple models, for example through hard voting. The Yaswanth et al. (2026) study summarized in the supplied review combines Logistic Regression, SVM, Random Forest, XGBoost and LSTM and adds risk scoring, SHAP/LIME explanations, reporting and adaptive retraining. Ensembles may combine complementary strengths, but increase model and validation complexity. Explainability can help users understand predictions, but it does not itself prove that a prediction is correct.

IV. Comparative Analysis of Existing Techniques

Table II compares the techniques discussed in the reviewed studies in terms of how they work, their strengths, and their weaknesses.

TABLE II

COMPARISON OF EXISTING TECHNIQUES

Technique	How it works	Strengths	Weaknesses
Blacklist	Matches URL/domain against known malicious entries	Simple and fast lookup	New or unreported URLs may be missing; depends on freshness
Rules / heuristics	Applies fixed suspicious-pattern rules	Easy to understand	Rules can be evaded; may flag legitimate sites
Logistic Regression	Learns a linear boundary from features	Fast and relatively interpretable	May miss complex nonlinear relationships
Decision Tree / Random Forest	Learns feature-based rules; Random Forest combines trees	Can model nonlinear patterns; feature importance may be available	Overfitting or feature/data dependence; forests are larger
SVM	Learns a separating boundary	Useful for high-dimensional classification	Can be computationally costly at scale
Naive Bayes	Probabilistic classifier often used with text features	Fast	Feature-independence assumption can be limiting
XGBoost / boosting	Combines sequentially trained trees	Strong nonlinear tabular modelling	Tuning and compute can increase complexity
CNN / LSTM / BiLSTM	Learns local and sequential character/token patterns	Can learn representations from sequences	More complex; may be harder to explain and require more compute
Hybrid / ensemble	Combines models or evidence types	Potentially complementary performance	More complex validation, deployment and maintenance
SHAP / LIME	Estimates feature contributions to predictions	Can make predictions more interpretable	Adds explanation complexity; explanations need careful interpretation

V. Research Gaps Identified

The following gaps are drawn from the nine studies as synthesized in the supplied literature review. They should be treated as research opportunities rather than claims that no prior work has addressed them.

Gap 1 — Newly Created and Zero-Day Phishing

Blacklist and fixed-rule approaches rely on previously known indicators or fixed patterns. The module should investigate whether learned URL patterns can identify suspicious URLs beyond a known list, while avoiding unsupported claims of zero-day detection without suitable testing.

Gap 2 — Dependence on Manually Engineered Features

Several traditional ML approaches depend on predefined features. NLP, TF-IDF, CountVectorizer and character-level representations offer alternative ways to encode URLs. The module can compare engineered features with text/character representations.

Gap 3 — Limited Feature Diversity

URL-only models may miss domain or webpage evidence. Domain-aware and hybrid studies add registration, DNS, HTML or behavioral information. The module should define which feature sources are actually available and measure whether added evidence justifies its collection cost.

Gap 4 — Generalization to Unseen Websites

High performance on one dataset does not automatically establish performance on unseen domains or independent sources. Use domain-aware splits and, where feasible, an independent dataset. Do not describe a source-based split as a temporal evaluation unless trustworthy timestamps exist.

Gap 5 — False Positives and False Negatives

Incorrectly flagging legitimate URLs can reduce usability, while missed phishing URLs can be harmful. The low phishing-class F1 reported by Renjith P N et al. (2025), despite 90% overall accuracy, illustrates why class-specific metrics matter. Report precision, recall, F1, confusion matrix and false-positive rate.

Gap 6 — Real-Time Detection

Many studies evaluate models offline. A practical module should measure feature extraction and prediction latency and avoid expensive enrichment unless it provides measurable value.

Gap 7 — Explainability

Deep models can be difficult to interpret. The supplied review notes SHAP and LIME as possible explanation methods. The module can expose important feature indicators where supported and evaluate whether explanation output is understandable.

Gap 8 — Continuous Adaptation

Phishing tactics evolve. Controlled retraining with newly labelled examples may help, but updates should be validated before deployment to avoid degrading performance.

Gap 9 — Dataset Quality and Comparability

Dataset source, time period, class balance, labels, duplicates and related domains affect reported results. Record provenance and preprocessing; avoid directly ranking headline accuracy values from studies with different datasets and protocols.

VI. Proposed System and Scope

The supplied literature review proposes a pipeline: URL input → validation/normalization → feature extraction → preprocessing/vectorization → trained classifier or ensemble → phishing/legitimate result → confidence/risk score → explanation/report → controlled retraining. A staged design is recommended so that the baseline is measured before more complex features and models are added.

A. Proposed Feature Groups

TABLE III

PROPOSED FEATURE GROUPS

Feature group	Examples
Lexical URL	URL length, token length, dots, hyphens, digits and special characters
Addressing	IP-address presence and subdomain count
Text / NLP	Suspicious keywords, TF-IDF and CountVectorizer
Domain	Domain age/lifetime, registration and DNS information where available
Structural / HTML	HTML characteristics and metadata, only if safely collected
Sequential	Character/token sequences and embeddings
Similarity	Similarity to known phishing patterns
Behavioral	Redirects or safely observable behavior, if supported

B. Staged Implementation

- Inspect labelled phishing and legitimate URL datasets; document sources, label definitions, class balance and available timestamps.
- Validate and normalize URLs using a documented, deterministic pipeline.
- Extract lexical and text features; handle missing values and document vectorization.
- Train and evaluate a traditional baseline before adding more complex models.
- Evaluate a character/sequence model if the available runtime and dependencies support it.
- Test an ensemble only if the individual models and validation results justify it.
- Add domain or webpage signals only if they can be collected safely and their benefit can be measured.
- Select decision thresholds on validation data; report recall and false-positive trade-offs.
- Record configuration, metrics, runtime and limitations; validate any retraining before replacing a model.

VII. Requirement Analysis

A. Functional Requirements

TABLE IV

FUNCTIONAL REQUIREMENTS

ID	Functional requirement
FR-1	Accept a website URL as input.
FR-2	Validate and normalize the submitted URL.
FR-3	Extract lexical URL features, including length, punctuation, digits, suspicious tokens, subdomains and IP-address presence.
FR-4	Optionally obtain domain registration and DNS features where available and safe.
FR-5	Optionally obtain webpage/HTML metadata if this capability is implemented.
FR-6	Preprocess numerical, categorical and textual features consistently.
FR-7	Apply feature selection, TF-IDF, CountVectorizer or learned representations as appropriate to the chosen model.
FR-8	Classify a URL as Phishing or Legitimate using a trained model or validated ensemble.
FR-9	Return a confidence/probability/risk score where supported by the model.
FR-10	Display the result in a user-friendly form.
FR-11	Provide important feature indicators or SHAP/LIME explanations if implemented.
FR-12	Support controlled logging/auditing while limiting unnecessary storage of submitted URLs.
FR-13	Support model updating/retraining with newly labelled examples through a controlled process.

B. Non-Functional Requirements

TABLE V

NON-FUNCTIONAL REQUIREMENTS

ID	Non-functional requirement
NFR-1 — Predictive quality	Evaluate on an independent held-out set; report accuracy together with precision, recall, F1 and confusion matrix.
NFR-2 — Precision	Minimize false alarms on legitimate websites.
NFR-3 — Recall	Minimize missed phishing websites.
NFR-4 — Balanced evaluation	Use F1 and class-specific metrics; include PR-AUC or another suitable score-based metric where available.
NFR-5 — Response time	Measure end-to-end prediction latency and aim for interactive response; set a numeric target after profiling on the target environment.
NFR-6 — Reliability	Produce reproducible behavior for the same model version and input conditions.
NFR-7 — Security	Treat URLs as untrusted input. Do not automatically open submitted URLs in the prediction path; isolate any URL-fetching or webpage-analysis component.
NFR-8 — Scalability	Support increasing URL volumes and measure batch throughput where relevant.
NFR-9 — Maintainability	Separate data processing, feature extraction, model inference, interface and reporting into testable components.
NFR-10 — Usability	Present a clear verdict and explain the meaning and limitations of any risk score.
NFR-11 — Explainability	Expose major decision factors where feasible, without presenting explanations as proof of correctness.
NFR-12 — Adaptability	Allow retraining or model replacement after validation as phishing patterns evolve.

C. Dataset Requirements

- Binary labels for the main task: legitimate and phishing.
- Multiple sources where licensing and access permit.
- Time diversity where collection dates are trustworthy.
- Domain-aware train/test splitting to reduce leakage.
- Documented class distribution and handling of imbalance.
- Independent test data from another source or period where feasible.
- Recorded provenance, collection date, labels and preprocessing decisions.
- Documented handling of inaccessible or expired pages if webpage features are used.

VIII. Evaluation Plan

The evaluation should use the same test split for each model being compared. Accuracy is useful but should not be the only reported metric. Precision measures how many predicted phishing URLs are truly phishing; recall measures how many phishing URLs are detected; F1 balances precision and recall; the confusion matrix shows true positives, true negatives, false positives and false negatives. AUC/PR-AUC can be used for

score-based evaluation where applicable, and latency should be measured for real-time use.

- Report dataset source, size, label definitions, class balance and preprocessing.
- Document whether the split is random, registered-domain-disjoint, source-separated or temporal.
- Report accuracy, precision, recall, F1, confusion matrix and suitable AUC metric where available.
- Measure latency and record the test environment.
- Choose classification thresholds using validation data rather than optimizing on the final test set.
- Use independent data or domain-aware evaluation where feasible; do not claim temporal generalization without trustworthy timestamps.
- Report limitations and any metrics that are not available as NR rather than inventing values.

IX. Conclusion

The nine studies in the supplied review show a progression from blacklists and rules to traditional machine learning, NLP/domain-aware methods, deep sequence models, hybrid ensembles and explainable AI. They also show that reported accuracies are not directly comparable because datasets, feature sets, class definitions and evaluation protocols differ. The most relevant requirements for the proposed module are diverse and documented data, consistent preprocessing, model comparison, independent/domain-aware evaluation, precision and recall reporting, efficient inference, explainability where feasible, and controlled updating. The central research opportunity is to build and evaluate a detector that is not judged solely by a single accuracy value, but by generalization, false-alarm cost, usability and reproducibility.

References

[1] Gujar, S. S. (2024). "Machine learning algorithms for detecting phishing websites." Proceedings of the International Conference on Smart Electronic Systems (ICSES).

[2] Kamble, N., & Mishra, N. (2024). "Securing cyberspace: Unveiling phishing attacks through deep neural networks for enhanced detection." Proceedings of the ASSIC Conference.

[3] Krishnaveni, A., & Balamurugan, S. (2024). "Phishing attack prediction using several machine learning techniques." Proceedings of the International Conference on Smart Electronic Systems (ICSES).

[4] Lakshmanarao, A., Surya Prabhakara Rao, P., & Bala Krishna, M. M. (2021). "Phishing website detection using novel machine learning fusion approach." Proceedings of the International Conference on Artificial Intelligence and Smart Systems (ICAIS).

[5] Parameshwar Patil, S. S. S., et al. (2020). "Phishing detection using Random Forest, SVM and Neural Network with backpropagation."

[6] Patel, P., Raj, R., & Sapra, P. (2025). "Phishing URLs detection using machine learning models." Proceedings of the RCSM Conference.

[7] Renjith, P. N., et al. (2025). "A novel deep learning module for accurate phishing website detection and prevention." Proceedings of the RCSM Conference.

[8] Sreenivasan, S., Ramachandran, P., Rout, A., & Pal, T. K. (2024). "Enhancing phishing detection through advanced machine learning techniques." Proceedings of the International Conference on Smart Electronic Systems (ICOSEC).

[9] Yaswanth, M. S., et al. (2026). "Multi model approach for phishing website detection using ML and DL techniques." Proceedings of the International Conference on System, Society and Information Technologies (ICSSIT).

Source note: This assignment is based on the supplied PDF, "A Smart Machine Learning Approach for Phishing Website Detection: A Source-Based Review of Nine Studies (2020–2026)." Bibliographic details above are reproduced from that review; verify full publication metadata against the original papers before final academic submission.